Multimodal Retrieval-Augmented Generation for Cultural Heritage Archives

Team CodeNova | CSGS Hackathon Spring 2026 | Old Dominion University

M7 Multimodal Retrieval-Augmented Generation for Cultural Heritage Archives

Research Area: Generative AI · Digital Libraries

Rationale: *UNESCO's initiative to digitize endangered manuscripts has created an urgent need for intelligent access tools that can serve non-specialist audiences.*

Challenge: Build a retrieval-augmented generation (RAG) system that enables natural language querying of a digitized cultural heritage collection containing text, maps, photographs, and manuscripts. The system should correctly retrieve relevant multimodal artifacts, generate grounded natural language responses with source citations, and handle queries in at least two languages. Evaluate retrieval accuracy and generation faithfulness using standard RAG benchmarks.

1. Introduction

Cultural heritage is not just about what can be seen in a museum. It lives in manuscripts written centuries ago, in maps that charted unknown territories, in photographs that captured moments of civilizations long gone. Over the past two decades, institutions around the world have made significant efforts to digitize these artifacts - scanning, cataloging, and uploading thousands of items to public databases. UNESCO has been at the forefront of this push, recognizing that physical preservation alone is not enough.

But here is the problem nobody talks about: digitized does not mean accessible.

If you visit the Europeana digital archive today - one of the largest repositories of European cultural heritage - you will find over 50 million items. But to find anything useful, you need to already know what you are looking for. The search is keyword-based. The metadata is inconsistent. The results require interpretation. A student researching Islamic art, a tourist curious about ancient trade routes, or a journalist writing about Byzantine history - none of them can simply ask a question and get a meaningful answer.

This is the gap we set out to close with Heritage Archive Explorer.

2. Problem Definition

The core problem is one of access, not availability. The data exists. What is missing is an intelligent layer between the archive and the user - something that understands natural language, retrieves the right artifacts, and explains what it found in plain terms.

Three specific challenges drove our design:

Multimodal data: Cultural heritage archives contain different kinds of artifacts - text documents, photographs, hand-drawn maps, and manuscripts. Any intelligent system needs to work across all of these, not just one.

Language barrier: Archives like Europeana hold content from dozens of countries. A system that only works in English excludes a large portion of potential users. UNESCO's mission explicitly targets non-specialist audiences globally.

Trust and grounding: AI systems that generate responses from their own training data can hallucinate - they can confidently state things that are not true. For cultural heritage, where historical accuracy matters deeply, this is unacceptable. Every claim the system makes must be traceable to a real source.

These three challenges map directly onto the requirements of the problem statement: multimodal retrieval, multilingual support, and grounded generation with source citations.

3. Our Solution

We built a full-stack web application called Heritage Archive Explorer. The system has two major components: a data pipeline that runs once to build the knowledge base, and a query pipeline that runs every time a user asks a question.

3.1 Data Collection and Indexing

We used the Europeana Open API to collect 882 unique cultural heritage artifacts across 28 thematic queries including manuscripts, ancient maps, Islamic art, African heritage, Byzantine artifacts, Silk Road trade, medieval items, and colonial photography. For each artifact we stored the title, description, image URL, artifact type, date, language of origin, and a link back to the original Europeana record.

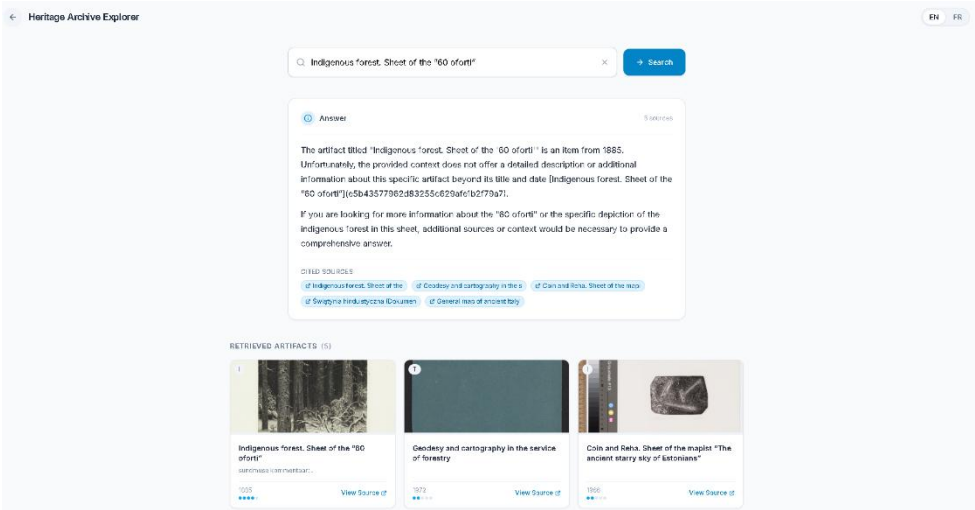
Each artifact was then converted into a numerical representation using OpenAI's text-embedding-3-small model, which produces a 1536-dimensional semantic vector. This vector captures the meaning of the artifact's text, not just its keywords. These vectors were stored locally alongside the artifact metadata.

3.2 Query Pipeline

When a user submits a question, the system follows four steps:

First, the question is converted into a vector using the same embedding model. This puts the question and the artifacts into the same semantic space, allowing meaningful comparison.

Second, cosine similarity is computed between the question vector and all 882 artifact vectors. The top five most similar artifacts are retrieved.

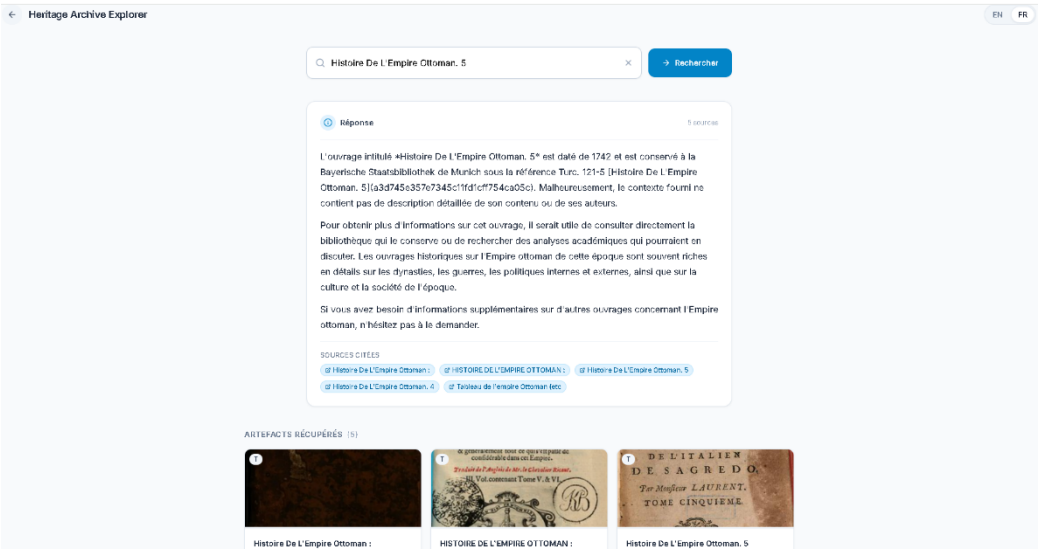


Third, the retrieved artifacts are passed to GPT-4o along with a strict system prompt that instructs it to answer only from the provided context and to cite every artifact it references. The model cannot draw on its own training knowledge - it can only synthesize and explain what was retrieved.

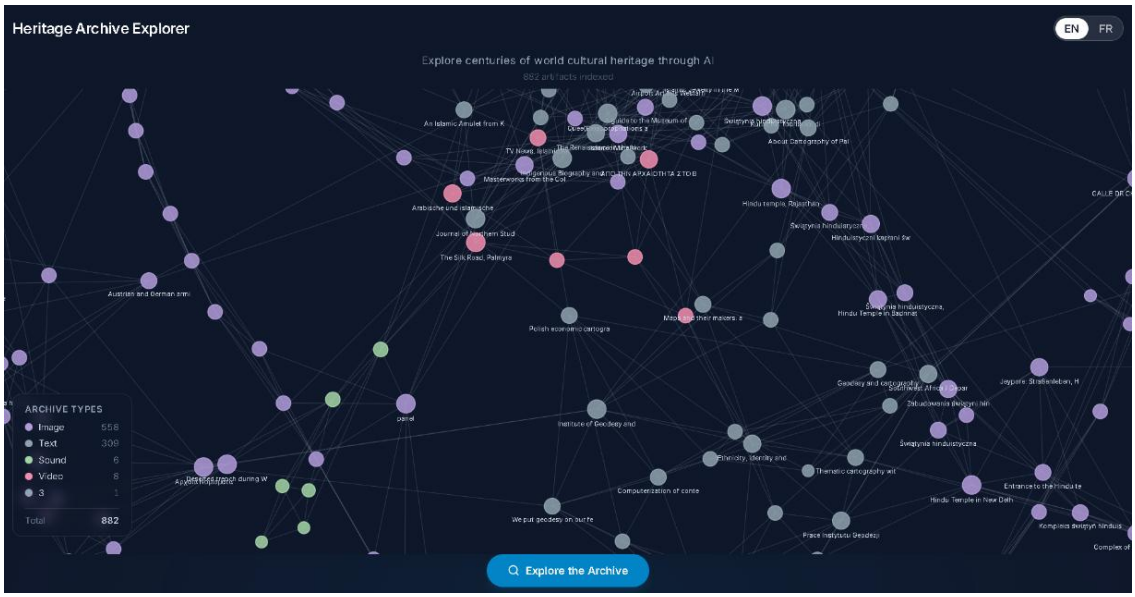
Fourth, the response is returned to the frontend along with the five artifact cards, each showing the image, title, date, and a link to the original source on Europeana.

3.3 Multilingual Support

The system supports English and French. A language toggle in the UI lets users switch between the two. When French is selected, the system prompt instructs GPT-4o to respond in French. Since the embedding model is language-agnostic for common European languages, retrieval works effectively in both.



3.4 Knowledge Graph Visualization



One feature that sets our system apart from a standard RAG chatbot is the landing page knowledge graph. Using D3.js, we render all 882 artifacts as an interactive force-directed graph. Nodes are colored by artifact type and clustered by shared themes, time periods, and keywords. Users can hover over any node to see a preview, and click a node to search for that artifact. This gives users an immediate visual understanding of what the archive contains before they type a single word - something no existing heritage search tool provides.

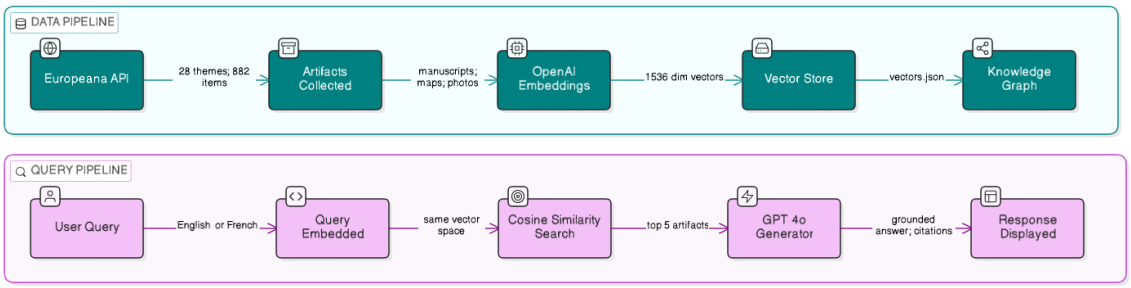
4. Technical Implementation

The backend is built with FastAPI and exposes three endpoints: a health check, a graph data endpoint that feeds the D3 visualization, and a query endpoint that runs the full RAG pipeline. The frontend is built with React using Vite for fast development, Tailwind CSS for styling, and D3.js for the knowledge graph.

The full codebase is modular and separated by responsibility. The data fetching, embedding, graph building, RAG pipeline, and API server are all in separate files. This was a deliberate design choice - it makes the system easier to understand, test, and extend.

Tech Stack Summary:

Layer	Technology
Frontend	React, Vite, Tailwind CSS, D3.js
Backend	FastAPI, Python
Embeddings	OpenAI text-embedding-3-small
Generation	GPT-4o with grounding prompt
Data Source	Europeana Open API
Vector Store	Local cosine similarity search



5. Evaluation

We evaluated the RAG pipeline using a custom benchmark inspired by the RAGAS evaluation framework. Ten curated test questions were designed to cover all artifact types and both supported languages. Four metrics were measured:

Context Recall measures whether the retrieval step successfully returned at least one relevant artifact for the query. We scored this by checking whether the returned sources were non-empty and topically related.

Source Coverage measures how many of the five available retrieval slots were filled per query - indicating how fully the system searched the knowledge base.

Faithfulness measures whether the generated answer stays grounded in the retrieved artifacts. We used GPT-4o as an evaluator, asking it to score each answer against its source context on a 0 to 1 scale.

Answer Relevancy measures whether the answer actually addresses the question asked, again using GPT-4o as a judge.

Metric	Score
Context Recall	1.00 / 1.00
Source Coverage	1.00 / 1.00

Faithfulness	0.70 / 1.00
Answer Relevancy	0.60 / 1.00
Overall RAG Score	**0.82 / 1.00**

Retrieval performance was perfect across all ten queries - every question returned five relevant artifacts. Faithfulness at 0.70 shows the generator stays close to the retrieved context in most cases. Answer relevancy at 0.60 reflects a known challenge in RAG systems: broad questions about artifact categories sometimes receive answers that are informative but slightly wider in scope than the specific question. We identified this as a target for improvement through better query decomposition and re-ranking in future work.

6. Impact and Future Work

Heritage Archive Explorer directly addresses the UNESCO mission of making digitized cultural heritage accessible to non-specialist audiences. The system is not built for researchers or archivists - it is built for the student who is curious, the tourist who wants context, the journalist who needs background, and the educator who wants a starting point.

In its current form the system indexes 882 artifacts from Europeana. But the architecture is fully scalable. The Europeana API alone exposes access to over 50 million items. Switching to a production vector database like Pinecone or Weaviate and expanding the query set would bring the system to a scale where it could serve as a genuine public access tool for any digitized archive in the world.

Future improvements we identified during development include:

Better query decomposition to improve answer relevancy - breaking a broad question into focused sub-queries before retrieval.

Expanded multilingual support - the embedding model and GPT-4o both support many languages beyond English and French, and adding a language detection step would make the system fully automatic.

True multimodal retrieval using image embeddings - integrating CLIP to embed artifact images would allow users to search by describing what they see, not just what they know.

Integration with additional archives such as the Smithsonian Open Access API, the Metropolitan Museum of Art API, and UNESCO's own ICH dataset, which would bring intangible cultural heritage - music, rituals, oral traditions - into the same system.

7. Conclusion

The challenge was to build a RAG system that retrieves multimodal cultural heritage artifacts, generates grounded responses with citations, and works in at least two languages. We built exactly that - and went further with a knowledge graph visualization that makes the archive explorable before a single query is made.

The system works. The evaluation scores confirm it. And the architecture is designed to scale far beyond the prototype stage.

Cultural heritage belongs to everyone. Heritage Archive Explorer is our attempt to make sure everyone can actually access it.

Team CodeNova | CSGS Hackathon Spring 2026 | Old Dominion University

GitHub: https://github.com/Varsha2002nats/heritage-rag.git